Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 06/10/2026**

A Mini Project Report on

CampusEats

A Canteen Pre-Order and Token System Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Dhruv Hemdev	2401036
Lakshya	_____
Yuvraj	_____

Under the guidance of

Prof. _____

Department of _____

_____ College of Engineering

2026–2027

CERTIFICATE

This is to certify that the mini project entitled **"CampusEats: A Canteen Pre-Order and Token System Delivered Through an Agile and DevOps Pipeline"** is a bona fide work of **Dhruv Hemdev (2401036)**, **Lakshya (_____)** and **Yuvraj (_____)**, submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide **Head of Department**

Date: _____ Place: _____

ABSTRACT

College canteens lose a large share of every short break to queueing: students wait to order, wait again for the food, and frequently return to class without eating. CampusEats addresses this with a web application in which a student orders from the classroom, receives a daily token number, follows the order through the kitchen in real time, and walks to the counter only when the order is ready. Kitchen staff work a live three-stage queue, and the canteen administrator maintains the menu and reviews sales through a dashboard.

The application was used as the vehicle for applying all seven experiments of the Agile Software Development & DevOps Lab to a single product. Work was planned as a Scrum backlog of fifteen issues across two sprints, each issue developed on its own Git branch and merged into `main` through a pull request. Both tiers of the application are containerised with Docker and composed with MySQL, Redis, Prometheus, Grafana and Nagios into a seven-service Docker Compose stack. A Jenkins multibranch pipeline lints, tests and builds every branch, and on `main` additionally deploys the stack, smoke-tests the exact version that was built, and rolls back to the last good version on failure. Prometheus and Grafana record business and service metrics over time, while Nagios checks the availability of every container and of the host machine through the NRPE agent configured in Experiment 7.

The delivered system comprises a React 19 frontend, a Node.js/Express 5 API covered by 42 automated tests at 92.9 % statement coverage, and nineteen consecutive successful Jenkins builds, nine of which deployed release versions 1.0.1 to 1.0.9 without manual intervention. A simulated lunch rush of 245 orders in four minutes was served at a median API latency of about 3.5 ms. A configuration defect in the frontend container health check was detected by monitoring after deployment and corrected through the same branch, pull request and pipeline flow, which illustrates the feedback loop that the DevOps practices are intended to create.

Keywords: Agile, Scrum, Jira, Git, Docker, Docker Compose, Jenkins, CI/CD, Prometheus, Grafana, Nagios, React, Node.js

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

A typical college break lasts fifteen to twenty minutes, and most students spend it at the canteen. Demand arrives in sharp peaks: within a few minutes of a lecture ending, a single counter receives more orders than the kitchen can prepare, and students queue first to order and then again to collect. The result is lost class time, uneven kitchen workload and no record of what was sold or when demand peaks.

The Agile Software Development & DevOps Lab introduced, through seven separate experiments, the practices used to deliver software of this kind reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment was performed in isolation on a toy workload. This mini project applies all seven to one realistic product so that their interaction —rather than each tool on its own—becomes visible.

1.2 Problem Statement

To design, build and operate a web application that lets students pre-order canteen food and collect it by token, and to deliver it through an Agile, fully automated pipeline in which every change is planned in Jira, versioned in Git, tested and built by Jenkins, deployed as containers and observed by monitoring tools.

1.3 Objectives

1. Allow students to register, browse the menu, place an order, receive a daily token and track the order live.
2. Give kitchen staff a live queue in which orders move through a fixed sequence of states, and allow dishes to be marked sold out instantly.
3. Give the administrator menu management and a sales dashboard showing revenue, peak hours and best-selling items.
4. Plan the work as a Scrum backlog with story points, sprints and a Definition of Done linked to the build pipeline.
5. Version every change on a branch named after its Jira issue and merge only through pull requests.
6. Containerise every component and run the full stack with one Docker Compose command.
7. Build, lint and test every branch automatically, and deploy `main` automatically with a smoke test and rollback.
8. Monitor business and service metrics over time and check the availability of every component and the host.

1.4 Scope

The system targets a single canteen with one kitchen. Payment is out of scope: orders are paid at the counter on collection, as is the practice today. Notifications are delivered through the live tracking page rather than SMS or push messages. The deployment target is a single host running Docker; the same Compose file and pipeline would apply to a college server.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in CampusEats	Section
1	Version control with Git	Branch per Jira issue, 15 pull requests, merge commits, release tag v1.0	5.1
2	Containerisation with Docker	Dockerfiles for the API (non-root, health check) and the frontend (multi-stage, Nginx)	5.2
3	Docker Compose	Seven-service stack with health-gated start-up, named volumes, one network	5.3
4	Jenkins CI pipeline	Multibranch declarative pipeline: install, lint, test (JUnit), build images	5.4
5	Automated deployment	Deploy on main, version-aware smoke test, automatic rollback	5.5
6	Agile lifecycle with Jira	Scrum project CE, 2 sprints, story points, issue keys in branches, commits and builds	2, 5.6
7	Monitoring	Prometheus + Grafana for metrics, Nagios + NRPE for availability	5.7

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Student	Order without queueing and collect at the right time	Menu and cart, live tracking, order history
Kitchen staff	Prepare orders in sequence and keep stock accurate	Kitchen queue board, stock panel
Administrator	Keep the menu correct and understand demand	Menu management, sales dashboard

Table 2.1: User roles

2.2 Functional Requirements

- Students register with name, email and password; staff and administrator accounts are seeded.
- The menu is grouped by category, searchable, marks vegetarian items and hides sold-out dishes.
- The order total is always computed by the server from current prices; a cart may hold at most ten of an item.
- Each order receives a token number that restarts at 1 every day according to Indian Standard Time.
- Orders move only along PLACED → PREPARING → READY → COLLECTED, or to CANCELLED before they are ready.
- Students see status changes within three seconds and may cancel only before cooking starts.
- The administrator can add, edit and delete dishes; changes are visible to students on the next request.
- The sales dashboard shows today’s revenue, orders, active orders and average preparation time, revenue for seven days, orders per hour and the top five items.

2.3 Non-Functional Requirements

- **Performance:** median API response under 50 ms during a lunch-hour peak; the menu, the most frequent request, is served from cache.
- **Security:** passwords stored as bcrypt hashes; JSON Web Tokens with an eight-hour expiry; every endpoint enforces its role.
- **Reliability:** every container has a health check; the database starts before the API; a failed release is rolled back automatically.
- **Observability:** request, latency and business metrics exposed for Prometheus; availability checked by Nagios.

- **Portability:** the complete system starts with a single `docker compose up` on any machine with Docker.

2.4 Scrum Setup

The project was organised as a Scrum project with the key CE in Jira Software Cloud, following the procedure of Experiment 6. The backlog is kept as data in scripts/jira/backlog.json and loaded into Jira through its REST API by scripts/jira/jira.mjs, which creates the issues, sets story points on the board, creates both sprints, links each issue to its pull request, records the Jenkins build that delivered it as a comment, and transitions issues as work completes. Keeping the backlog in the repository means the plan is versioned alongside the code it describes. The issue key, from CE-1 to CE-15, is written into the branch name, every commit message, the pull-request title and the Jenkins build description.

User-facing requirements were written as stories in the form “As a <role>, I want <capability>, so that <benefit>” with acceptance criteria, and estimated in story points. Technical work with no user-facing behaviour—setting up the repository, containers, pipeline and monitoring—was raised as tasks without points, as in Experiment 6. Jira can assign an issue only to a person with an account on the site, and all work was carried out from one account, so the owner’s name is written at the start of each issue title—for example “[Lakshya] Browse menu by category”—and added as a label, which lets the backlog be filtered by team member (Table 2.4).

2.5 Product Backlog

Key	Type	Summary	Pts	Owner	Sprint
CE-1	Story	Student sign up and login	5	Dhruv	1
CE-2	Story	Browse menu by category	3	Lakshya	1
CE-3	Story	Add to cart, place order and get a token	5	Yuvraj	1
CE-4	Story	Staff live order queue	5	Lakshya	1
CE-5	Task	GitHub repository and branching rules	—	Dhruv	1
CE-6	Task	Dockerfiles and docker-compose stack	—	Yuvraj	1
CE-7	Task	Jenkins CI pipeline: install, lint, test, build	—	Dhruv	1
CE-8	Story	Student live order tracking	3	Yuvraj	1
CE-9	Story	Student order history	3	Lakshya	2
CE-10	Story	Admin menu management	5	Dhruv	2
CE-11	Story	Admin sales dashboard	3	Yuvraj	2
CE-12	Task	Jenkins CD: automatic deployment of main	—	Lakshya	2
CE-13	Task	Prometheus metrics and Grafana dashboard	—	Dhruv	2
CE-14	Task	Nagios checks for the stack and host	—	Lakshya	2
CE-15	Bug	Frontend container reported unhealthy (health check on IPv6 localhost)	—	Yuvraj	2

Table 2.2: Product backlog of project CE

2.6 Sprint Plan

Sprint	Goal	Stories (points)	Tasks
CE Sprint 1	Students can order, get a token and track it; kitchen works the live queue; code is built and tested by Jenkins on every push.	CE-1 (5), CE-2 (3), CE-3 (5), CE-4 (5), CE-8 (3) = 21	CE-5, CE-6, CE-7
CE Sprint 2	Admin tools and order history; every merge to main is deployed automatically and the stack is monitored.	CE-9 (3), CE-10 (5), CE-11 (3) = 11	CE-12, CE-13, CE-14, bug CE-15

Table 2.3: Sprint goals and scope

Member	Stories	Points	Tasks / bugs
Dhruv Hemdev	CE-1, CE-10	10	CE-5, CE-7, CE-13
Lakshya	CE-2, CE-4, CE-9	11	CE-12, CE-14
Yuvraj	CE-3, CE-8, CE-11	11	CE-6, CE-15
Total	**8 stories**	**32**	**7 tasks, 1 bug**

Table 2.4: Distribution of work across the team

2.7 Definition of Done

An issue is moved to **Done** only when all of the following hold:

1. Every commit for the issue begins with its key, and the branch is named `feature/CE-<n>-<slug>` (or `fix/` for bugs).
2. The change reached `main` through a pull request whose Jenkins status check is green.
3. The Jenkins build of `main` that contains the change reported SUCCESS, which includes the deployment and smoke test.
4. The behaviour was verified in the running application.

The third condition is the link between the plan and the pipeline: an issue cannot be closed while the build that delivers it is red.

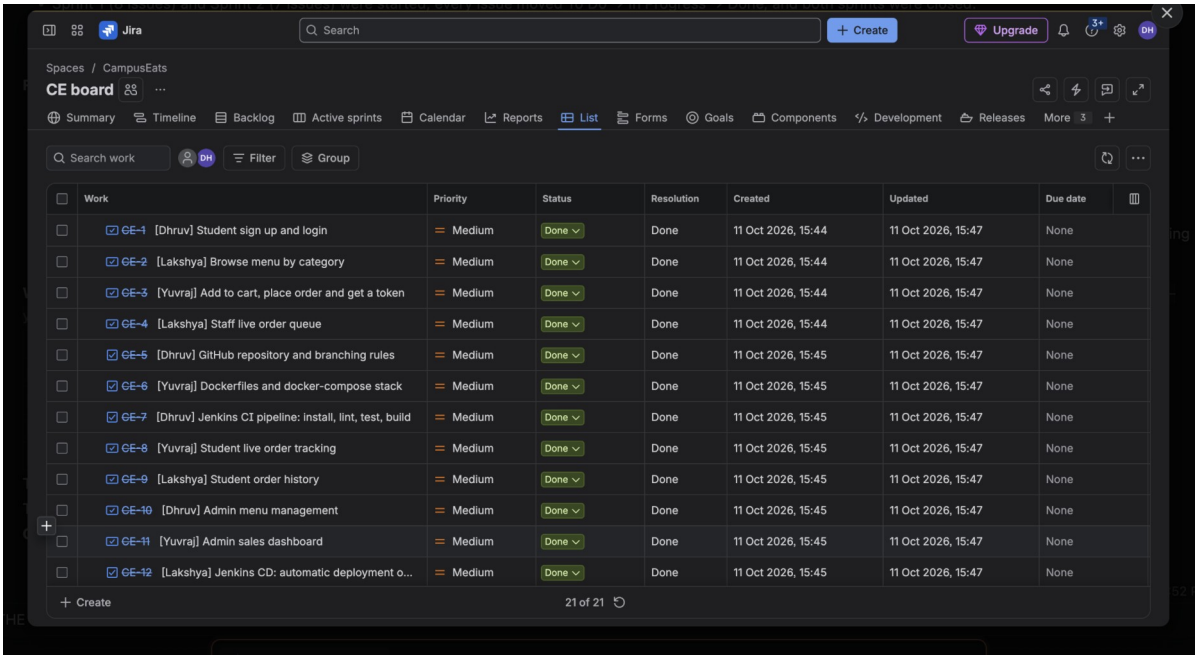


Figure 2.1: Issues of project CE with owners and status



Figure 2.2: Jira summary of project CE: work items by status, type and assignee, and the completion trend

3. System Design

3.1 Architecture

CampusEats is a three-tier web application packaged as containers (Figure 3.1). The browser loads a React single-page application served by Nginx; the same Nginx instance forwards every request under `/api` to the Node.js API, so the browser talks to a single origin. The API stores users, the menu and orders in MySQL and uses Redis for two purposes: caching the menu, which is read on every page load, and holding the daily token counter, which must be incremented atomically when many students order at once.

Three monitoring services run alongside the application in the same Compose network. Prometheus scrapes the API's `/metrics` endpoint every five seconds, Grafana queries Prometheus to draw the dashboard, and Nagios performs HTTP and TCP checks against each container by service name and queries the host through the NRPE agent. Outside the stack, Jenkins polls GitHub, builds every branch, and deploys `main` by running Docker Compose on the same host.

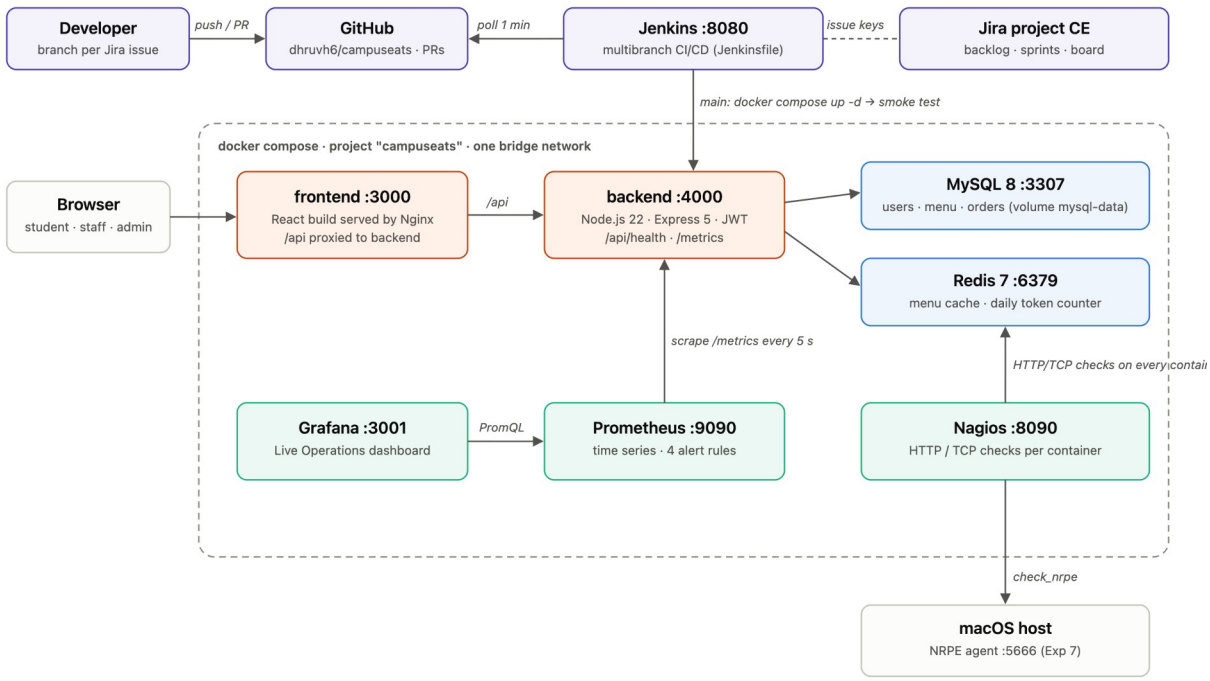


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	React 19, Vite 8, React Router 7	Component model suits three role-specific UIs; Vite builds in under a second
Web server	Nginx 1.29 (alpine)	Serves static files and proxies /api, so no CORS in production
API	Node.js 22, Express 5	Same language as the frontend; Express 5 forwards async

Layer	Technology	Reason for choice
		errors automatically
Authentication	bcryptjs, jsonwebtoken	Stateless sessions that work behind any number of API containers
Database	MySQL 8.0	Relational integrity between users, orders and order lines; used in Experiment 3
Cache	Redis 7	Atomic INCR for tokens; TTL-based menu cache
Testing	Jest 30, Supertest, Vitest 4	API tests without a database through an in-memory store; JUnit output for Jenkins
CI/CD	Jenkins 2.579 multibranch	Builds every branch from the Jenkinsfile in the repository (Exp 4, 5)
Monitoring	Prometheus, Grafana, Nagios Core 4.5	Time-series metrics and state-based availability checks (Exp 7)
Planning	Jira Software Cloud (Scrum)	Backlog, sprints, board and reports, as used in Experiment 6

Table 3.1: Technology stack

3.3 Database Design

Four tables hold the data (Figure 3.2). Each order belongs to one user and has many order lines. An order line copies the dish name and price at the time of ordering, so that editing or deleting a dish later never alters the history of past orders; for the same reason its reference to the menu is set to NULL rather than cascading when a dish is deleted. The order row records a timestamp for each state change, from which preparation time is derived. Indexes on `status` and `created_at` support the kitchen queue and the sales queries.

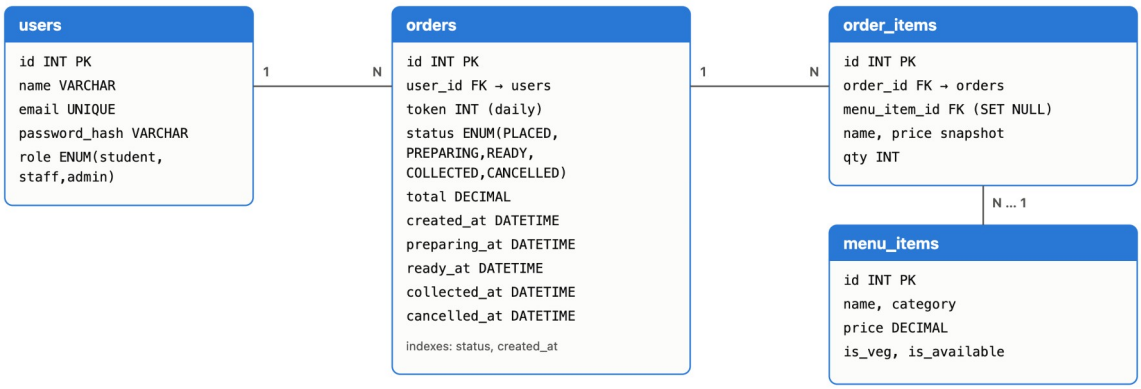


Figure 3.2: Entity-relationship diagram

3.4 Order Lifecycle

The order state machine (Figure 3.3) is enforced by the API rather than the user interface: the allowed transitions are held in a single table in `routes/orders.js`, and any other request is rejected with HTTP 409 Conflict. A student may cancel only while the order is PLACED; staff may also cancel while it is PREPARING.

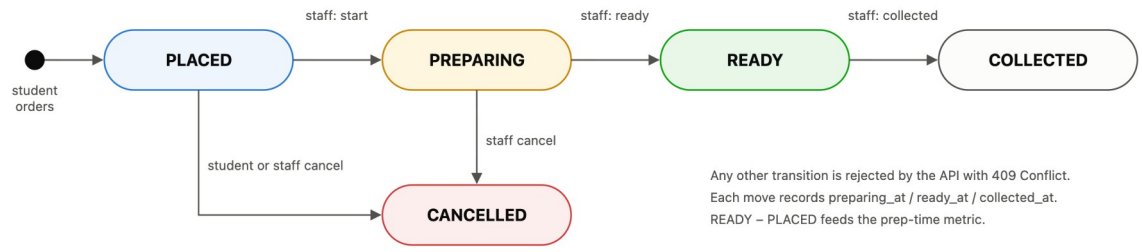


Figure 3.3: Order state machine

3.5 API Design

Method	Endpoint	Role	Purpose
POST	/api/auth/register	public	Create a student account
POST	/api/auth/login	public	Return a JWT
GET	/api/menu	public	Available dishes (Redis cached; X-Cache: HIT / MISS)
GET	/api/menu/all	staff, admin	All dishes including sold out
POST, PUT, DELETE	/api/menu[/:id]	admin	Manage dishes
PATCH	/api/menu/:id/availability	staff, admin	Mark sold out or back in stock
POST	/api/orders	student	Place order; returns token
GET	/api/orders/mine	student	Order history
GET	/api/orders/:id	owner, staff	Live order status
PATCH	/api/orders/:id/cancel	owner	Cancel while PLACED
GET	/api/orders/queue	staff, admin	Active orders
PATCH	/api/orders/:id/status	staff, admin	Advance the order state
GET	/api/admin/stats	admin	Dashboard figures
GET	/api/health	public	Database and cache check, version
GET	/metrics	Prometheus	Metrics in Prometheus text format

Table 3.2: REST API

3.6 Security Design

- Passwords are hashed with bcrypt (cost 10); the hash is never returned by any endpoint.
- Every protected route verifies a signed JWT and then checks the role, so a student token cannot reach kitchen or admin endpoints (HTTP 403).
- Prices and totals are taken from the database at order time; the client sends only item identifiers and quantities.
- A student requesting another student's order receives 404 rather than 403, so the existence of the order is not disclosed.

- The API container runs as the unprivileged `node` user; secrets such as the JWT key and database passwords are supplied through environment variables, and the repository's `.env` file is excluded from Git.

4. Application Implementation

The API is organised around a store interface with two implementations: `mysqlStore` for the deployed system and `memoryStore` for tests and local development. Route handlers depend only on the interface, which is what allows the 42 API tests to run in Jenkins in under a second without a database. The frontend is a single React application whose navigation and routes depend on the role in the logged-in user's token.

4.1 Student Experience

After logging in (Figure 4.1), a student sees the menu grouped by category with a search box, vegetarian markers and a running cart (Figure 4.2). Placing the order returns a token, and the tracking page (Figure 4.3) polls the API every three seconds, advancing a four-step tracker as the kitchen works. The order history (Figure 4.4) lists previous orders with their status and total.

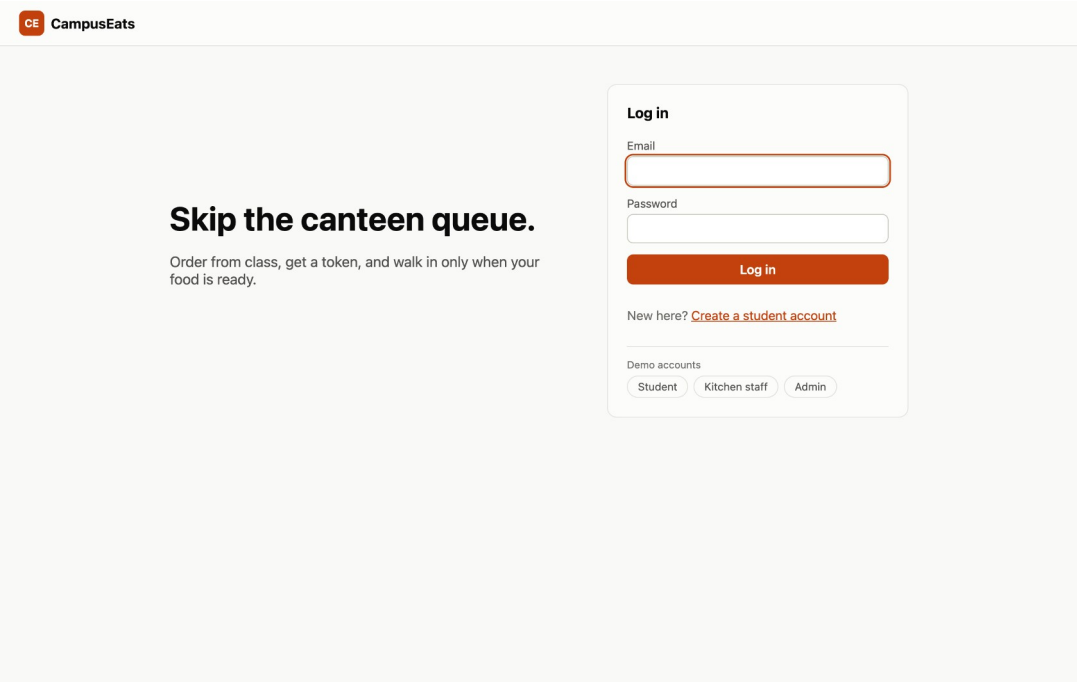


Figure 4.1: Login page with demo-account shortcuts

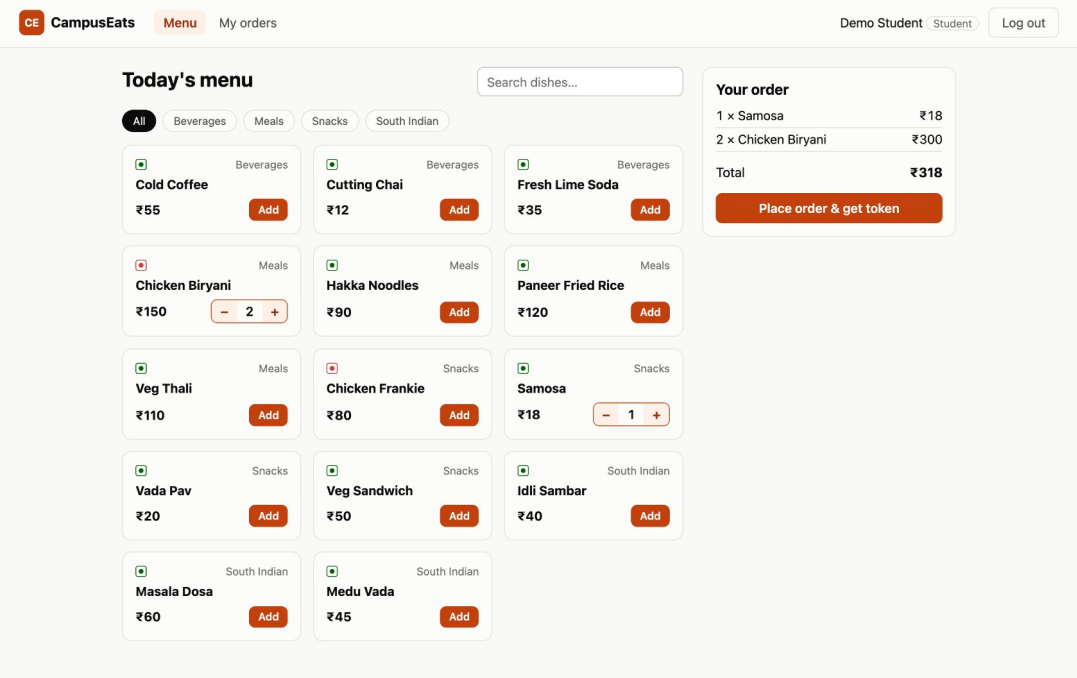


Figure 4.2: Menu with category filter and cart

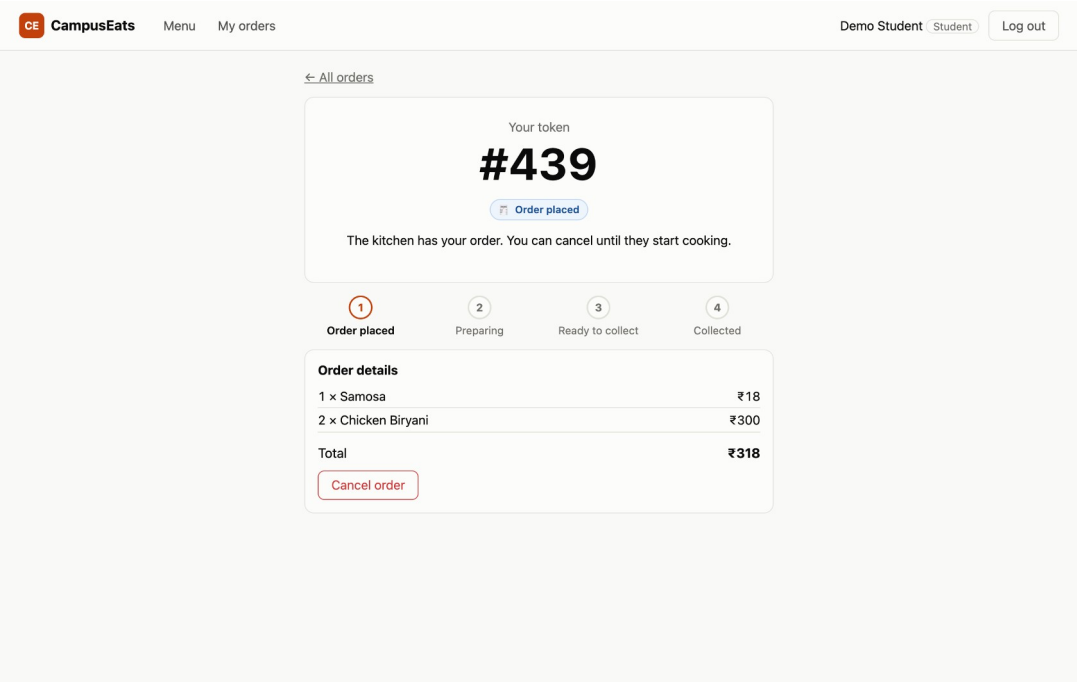


Figure 4.3: Live order tracking with token

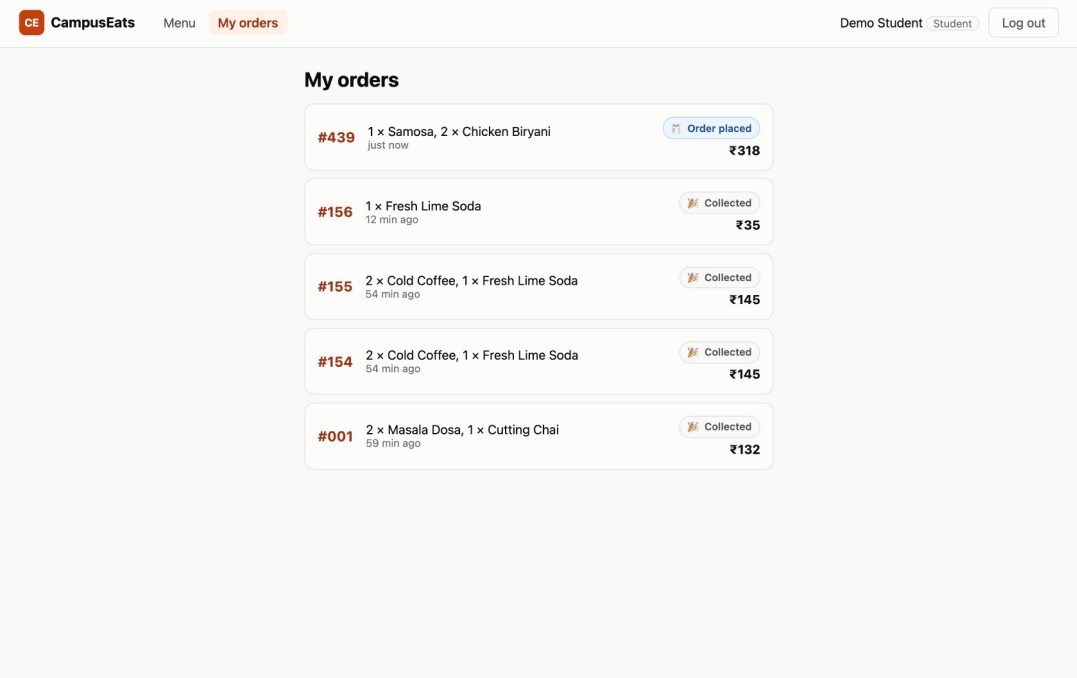


Figure 4.4: Order history

4.2 Kitchen Queue

Staff work a three-column board—New, Preparing and Ready for pickup—refreshed every three seconds (Figure 4.5). Each ticket shows the token, the student and the items, with a single action that advances it to the next state. Below the board a stock panel marks dishes sold out; the change clears the Redis menu cache, so students stop seeing the dish on their next request. Figure 4.5 was captured during a simulated lunch rush with 37 active orders.

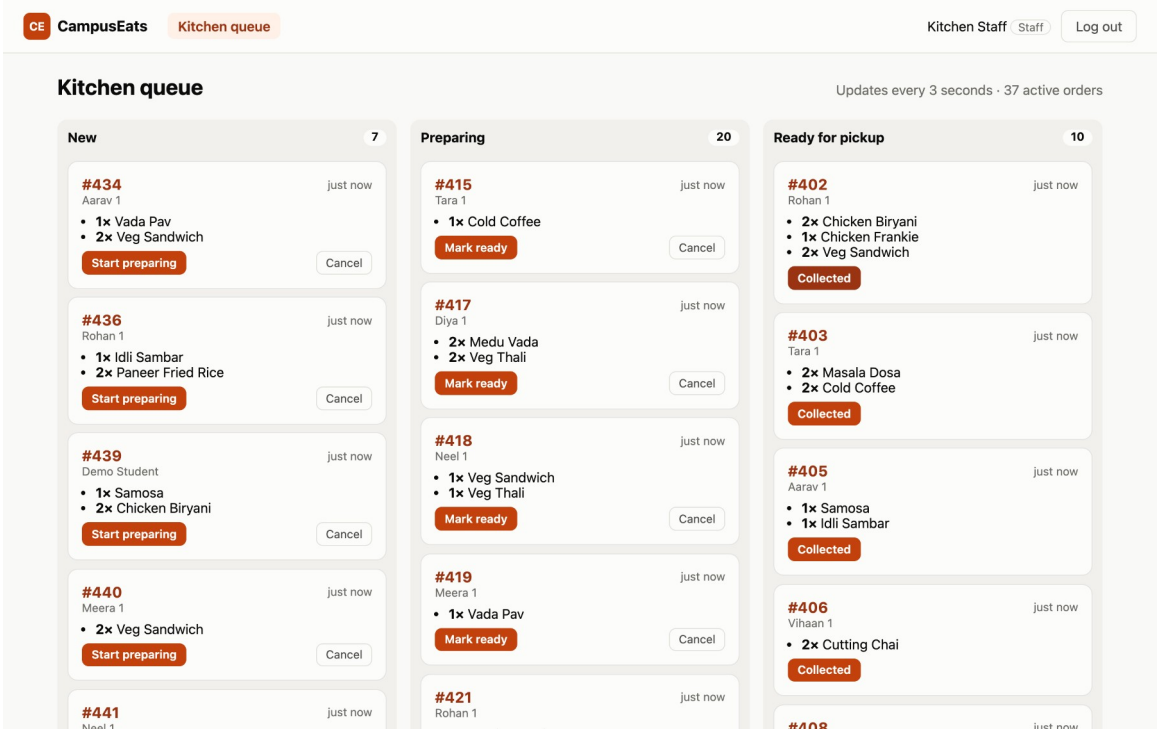


Figure 4.5: Kitchen queue during a simulated lunch rush

4.3 Administrator Tools

The sales dashboard (Figure 4.6) leads with today's revenue, followed by orders, active orders and average preparation time, a seven-day revenue chart, orders by hour and the five best-selling items. Every chart offers a hover tooltip and a table view, so values are never conveyed by colour or bar length alone. The figures are computed by a pure function, `salesStats`, which is unit-tested with fixed dates, including the case of an order placed just before and just after midnight IST. Menu management (Figure 4.7) supports adding, editing, marking sold out and deleting dishes.

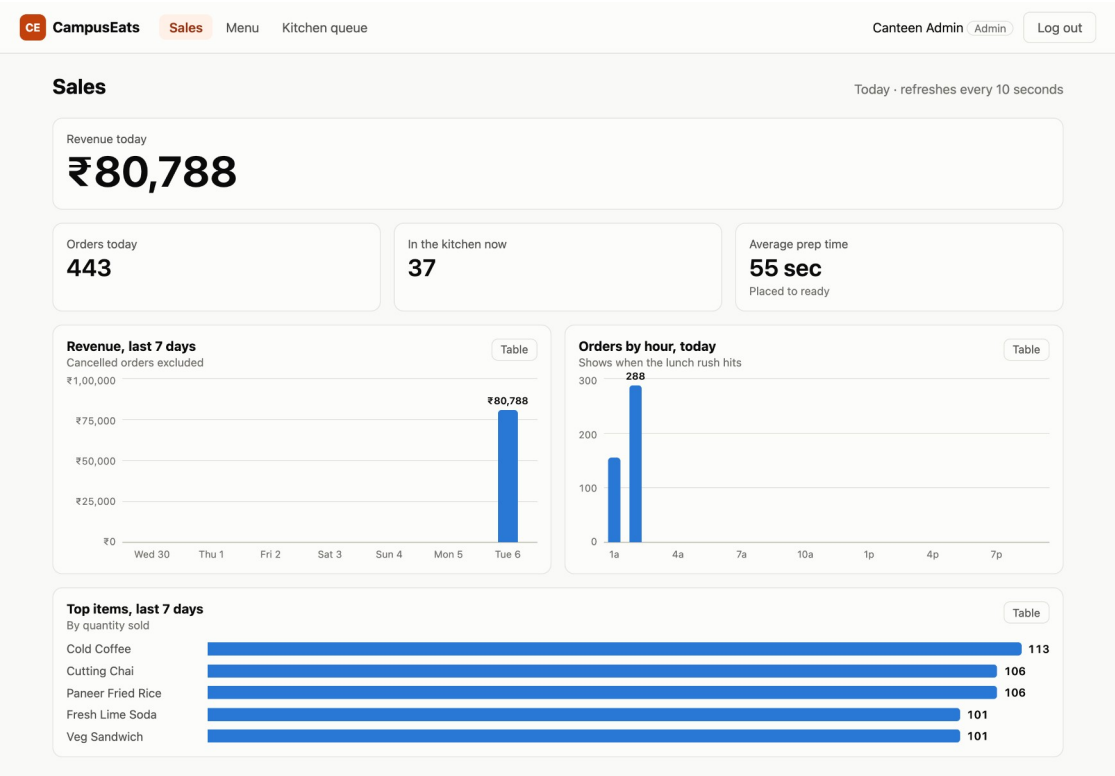


Figure 4.6: Admin sales dashboard

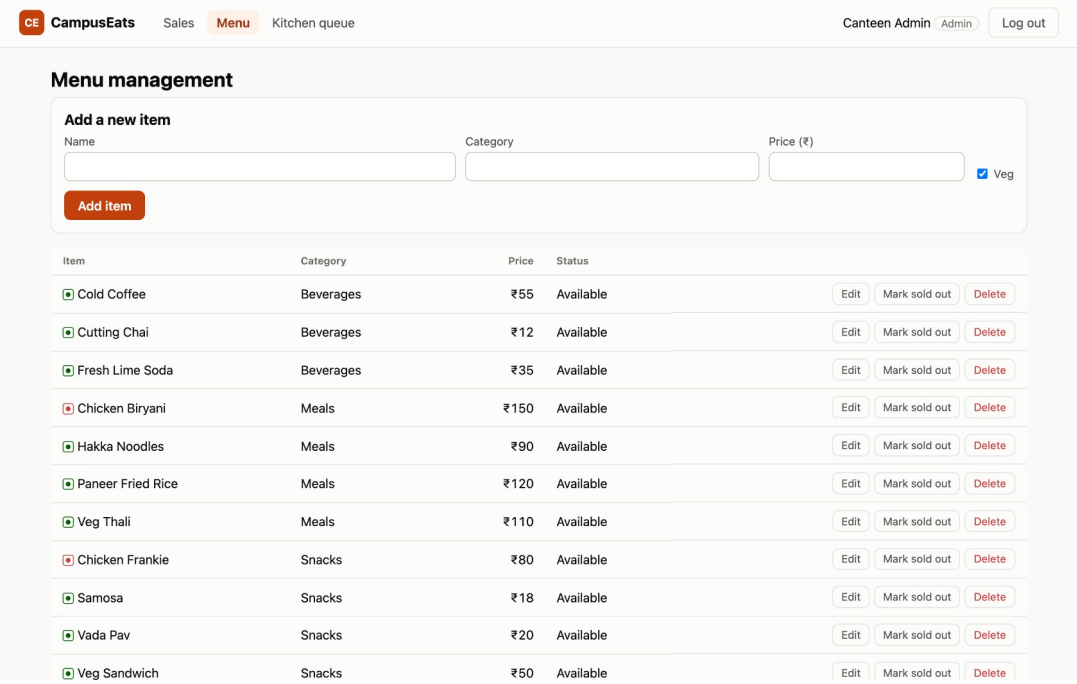


Figure 4.7: Admin menu management

4.4 Caching and Token Generation

The menu endpoint first reads the key `menu:available` from Redis; on a miss it queries MySQL and stores the result for 60 seconds. Every write to the menu deletes the key, so the cache never serves a stale menu after an administrator's change; the `X-Cache` response header exposes whether a request was a hit. Token numbers are produced by `INCR token:<IST date>`, which Redis executes atomically, so two students ordering in the same millisecond can never receive the same token. The key expires after 36 hours, and because the date is computed in the Asia/Kolkata time zone the sequence restarts at midnight on campus rather than midnight UTC.

4.5 Instrumentation

The API exposes metrics through `prom-client`. An Express middleware records a request counter and a latency histogram labelled by method, route pattern and status; labelling by route pattern (for example `/api/orders/:id`) rather than by URL keeps the number of time series bounded. Business metrics are recorded where the business events happen: orders placed, revenue, items sold by dish, preparation time when an order becomes READY, and a gauge of the kitchen queue by state that is read from the database each time Prometheus scrapes.

5. DevOps Implementation

This chapter describes how each lab experiment was applied. Figure 5.1 summarises the delivery pipeline that ties them together: a change starts on a branch named after its Jira issue, is built and tested by Jenkins, reaches `main` through a pull request, and is then deployed, smoke-tested and monitored.

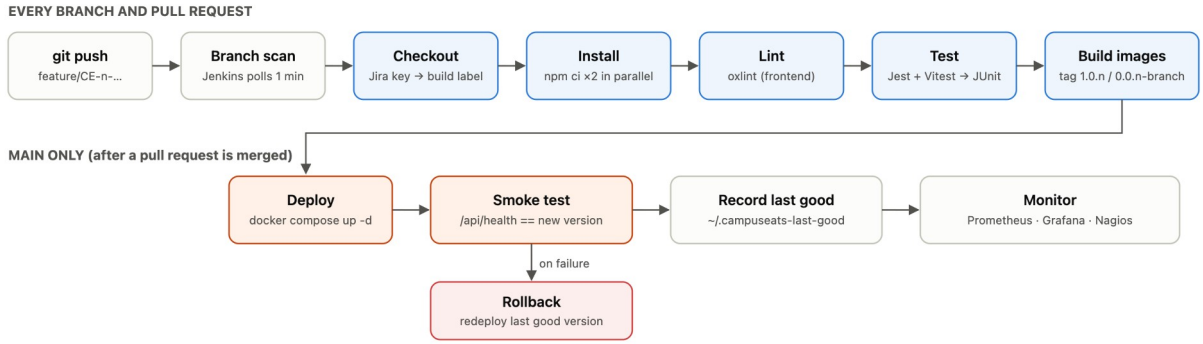


Figure 5.1: Delivery pipeline from push to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

The repository `dhruvh6/campuseats` is hosted privately on GitHub. The branching rule agreed in issue CE-5 is one branch per Jira issue, named `feature/CE-<n>-<slug>` for stories and tasks and `fix/CE-<n>-<slug>` for bugs, with every commit message beginning with the issue key. Changes reach `main` only through pull requests merged with a merge commit, which keeps each issue's commits grouped in the history. The commit author records the team member who owned the issue; the merge commits are made by the repository account.

PR	Title	Owner
#1	CE-1 Student sign up and login	Dhruv
#2	CE-2 Browse menu by category	Lakshya
#3	CE-3 Add to cart, place order and get a token	Yuvraj
#4	CE-4 Staff live order queue	Lakshya
#5	CE-8 Student live order tracking	Yuvraj
#6	CE-6 Dockerfiles and docker-compose stack	Yuvraj
#7	CE-7 Jenkins CI pipeline: install, lint, test, build	Dhruv
#8	CE-12 Jenkins CD: automatic deployment of main	Lakshya
#9	CE-9 Student order history	Lakshya
#10	CE-10 Admin menu management	Dhruv
#11	CE-11 Admin sales dashboard	Yuvraj
#12	CE-13 Prometheus metrics and Grafana dashboard	Dhruv
#13	CE-14 Nagios checks for the stack and host	Lakshya
#14	CE-5 Project README	Dhruv

PR	Title	Owner
#15	CE-15 Fix frontend container reported as unhealthy	Yuvraj

Table 5.1: Pull requests merged into main

The resulting history, abbreviated, shows each issue's commits on its own branch followed by the merge:

```
$ git log --oneline --graph
*   f19c6b8 Merge CE-15: frontend healthcheck fix
|\
| * e39ed34 CE-15 Fix frontend healthcheck failing on IPv6 localhost
|/
*   e93384e Merge CE-5: project README
*   ac8168f Merge CE-14: Nagios checks for the stack and host
*   24915a8 Merge CE-13: Prometheus metrics and Grafana dashboard
...
*   5f97de3 Merge CE-7: Jenkins CI pipeline
|\
| * ca91510 CE-7 Tag feature-branch images separately from main releases
| * 358f3b6 CE-7 Jenkins multibranch CI pipeline
|/
...
```

At the end of Sprint 2 the commit at the head of `main` was tagged `v1.0` and published as a GitHub release. Should a release need to be undone after it has been shared, `git revert` of its merge commit creates a new commit that reverses it without rewriting history—the non-destructive rollback practised in Experiment 1—and the pipeline then deploys the reverted state like any other change.

5.2 Containerisation with Docker (Experiment 2)

Each tier has its own Dockerfile. The API image installs production dependencies in a layer of its own, so the slow `npm ci` step is reused from cache whenever only source code changes:

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY src ./src
ARG APP_VERSION=dev
ENV NODE_ENV=production APP_VERSION=${APP_VERSION} PORT=4000
USER node
HEALTHCHECK --interval=15s --timeout=3s --start-period=20s --retries=3 \
  CMD wget -qO- http://localhost:4000/api/health || exit 1
CMD ["node", "src/server.js"]
```

The frontend uses a multi-stage build. The first stage, based on Node, installs all dependencies and runs `vite build`; the second stage starts from `nginx:alpine` and copies only the compiled files and the Nginx configuration. The build toolchain never reaches the final image, which is 92 MB against 267 MB for the API. The version string is passed as a build argument and reported by `/api/health`, which is how the pipeline later proves which build is running.

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

A single `docker-compose.yml` defines seven services under the project name `campuseats` (Table 5.2). Compared with Experiment 3, two refinements were needed for a real application. First, the API must not start before MySQL accepts connections, so MySQL and Redis declare health checks and the backend uses `depends_on` with `condition: service_healthy`. Second, the MySQL schema is applied on first start by mounting `db/init.sql` into the image's initialisation directory, and the data lives in the named volume `mysql-data`, so orders survive redeployments.

Service	Image	Host port	Notes
mysql	mysql:8.0	3307	Health check, schema on first start, volume mysql-data
redis	redis:7-alpine	6379	Health check
backend	campuseats-backend:<version>	4000	Starts only when mysql and redis are healthy
frontend	campuseats-frontend:<version>	3000	Nginx serving React and proxying /api
prometheus	prom/prometheus	9090	Scrape config and alert rules mounted read-only
grafana	grafana/grafana	3001	Provisioned datasource and dashboard
nagios	jasonrivers/nagios	8090	Object config mounted; amd64 image emulated on Apple Silicon

Table 5.2: Services in the Compose stack

Host port 3307 is used for MySQL because a MySQL server installed on the host already holds 3306, the same situation met in Experiment 3. The API is published on 4000 because macOS reserves 5000 for AirPlay, and Nagios on 8090 because Jenkins occupies 8080.

5.4 Continuous Integration with Jenkins (Experiment 4)

Jenkins runs a **multibranch** pipeline job named CampusEats. Instead of building one fixed branch, it scans the GitHub repository every minute, creates a job for every branch that contains a `Jenkinsfile`, and builds each branch whenever its head commit changes. The pipeline is declarative, as in Experiment 4, and is stored in the repository so that it is versioned and reviewed like any other code.

Stage	Steps	Purpose
Checkout	checkout scm; extract CE-n from branch name or commit message	Label the build with its Jira issue
Install	npm ci for backend and frontend, in parallel	Reproducible dependencies from lock files
Lint	oxlint on the frontend	Catch React and JavaScript mistakes before tests
Test	Jest (42 API tests) and Vitest (8 UI tests) in parallel	JUnit reports published to Jenkins
Build images	docker compose build backend frontend	Images tagged with the build version
Deploy*	docker compose up -d --no-build --remove-orphans	Run the images that were just tested

Stage	Steps	Purpose
Smoke test*	Wait for /api/health to report the new version; check UI and /api via Nginx	Prove the new build is live

Table 5.3: Pipeline stages (main only, added in CE-12)*

The first build of branch `feature/CE-7-jenkins-ci` exposed a design flaw: every branch tagged its images `1.0.<build number>`, so a feature branch's build number 3 would overwrite the release image of `main` build number 3—precisely the image a rollback would need. The fix, made as a second commit on the same branch before it was merged, gives release builds on `main` the tag `1.0.<n>` and every other branch `0.0.<n>-<branch>`:

```
environment {
    // main -> 1.0.<n>;  any other branch -> 0.0.<n>-<branch>
    APP_VERSION = "${env.BRANCH_NAME == 'main'
        ? '1.0.' + env.BUILD_NUMBER
        : '0.0.' + env.BUILD_NUMBER + '-' +
          env.BRANCH_NAME.toLowerCase().replaceAll('[^a-z0-9]+', '-')}"
}
```

Because Jenkins runs on a developer machine it cannot receive GitHub webhooks, so the result of each build was published to the pull request as a GitHub commit status named `jenkins/campuseats`; a green check on the pull request was the condition for merging. Console output from a feature-branch build:

```
Commit 358f3b6 by Dhruv Hemdev: CE-7 Jenkins multibranch CI pipeline
Jira issue for this build: CE-7
+ npm ci --no-audit --no-fund        added 58 packages in 726ms
+ npm ci --no-audit --no-fund        added 401 packages in 1s
 Test Files  1 passed (1)       Tests  8 passed (8)
Tests:       29 passed, 29 total
 Image campuseats-backend:1.0.1 Built
 Image campuseats-frontend:1.0.1 Built
SUCCESS - v1.0.1 (CE-7) passed on feature/CE-7-jenkins-ci
```

5.5 Automated Deployment (Experiment 5)

Issue CE-12 extended the pipeline with two stages that run only on `main`. **Deploy** runs `docker compose up -d --no-build --remove-orphans`, so the containers are recreated from exactly the images that passed the tests earlier in the same run. **Smoke test** then polls `/api/health` until it reports the version string of this build—a check that fails if an old container is still answering—and verifies that the frontend and the `/api` proxy respond through Nginx. On success the version is written to `~/.campuseats-last-good-version`; if a later `main` build fails, the `post { failure }` block redeploys that recorded version.

```
+ docker compose up -d --no-build --remove-orphans
 Container campuseats-backend  Recreated
 Container campuseats-frontend Recreated
+ curl -fs http://localhost:4000/api/health | grep -q "\"version\":\"1.0.2\""
API 1.0.2 is healthy
Frontend and API through Nginx are serving
```

Every merge after CE-12 was deployed without manual intervention. Table 5.4 lists the builds of `main`; feature-branch builds are summarised in Chapter 7.

Build	Version	Delivered issue	Result	Duration
#1	1.0.1	CE-7 CI pipeline (CI only)	SUCCESS	16 s
#2	1.0.2	CE-12 continuous deployment (first automatic deploy)	SUCCESS	31 s
#3	1.0.3	CE-9 order history	SUCCESS	23 s
#4	1.0.4	CE-10 admin menu	SUCCESS	21 s
#5	1.0.5	CE-11 sales dashboard	SUCCESS	22 s
#6	1.0.6	CE-13 Prometheus and Grafana	SUCCESS	21 s
#7	1.0.7	CE-14 Nagios	SUCCESS	24 s
#8	1.0.8	CE-5 README	SUCCESS	20 s
#9	1.0.9	CE-15 health check fix	SUCCESS	20 s

Table 5.4: Release history of main

5.6 Integrating Jira with the Pipeline (Experiment 6)

Traceability runs through the issue key. A key such as CE-12 appears in the branch name, in every commit message, in the pull-request title, in the Jenkins build description (“v1.0.2 · Jira CE-12”) and in the build’s console output, so any of these can be traced to the others. In Jira each issue carries a link to its pull request and a comment recording the Jenkins build and release that delivered it; for CE-12 the comment reads “Jenkins main #2 deployed v1.0.2 automatically and the smoke test confirmed /api/health reported 1.0.2”. The Definition of Done (Section 2.7) requires that build to be green before the issue is moved to Done, so the state of the board follows the state of the pipeline. Both sprints were closed with all fifteen issues done and nothing carried over.

5.7 Monitoring (Experiment 7)

Experiment 7 contrasted two models of monitoring: the state-based model of Nagios, which asks whether something is broken now, and the time-series model of Prometheus and Grafana, which records how a value has behaved. The project uses both, each for what it does best.

5.7.1 Prometheus metrics and alerts

Metric	Type	Meaning
campuseats_http_requests_total	counter	Requests by method, route pattern and status
campuseats_http_request_duration_seconds	histogram	API latency; p50 and p95 derived in PromQL
campuseats_orders_total	counter	Orders placed
campuseats_revenue_rupees_total	counter	Value of orders placed
campuseats_items_sold_total	counter	Units sold, by dish
campuseats_order_prep_seconds	histogram	Time from PLACED to READY

Metric	Type	Meaning
campuseats_orders_in_queue	gauge	Active orders by state, read at scrape time
campuseats_process_*, campuseats_nodejs_*	default	CPU, memory, event-loop lag of the API process

Table 5.5: Metrics exposed by the API

Alert	Condition	Severity
CampusEatsApiDown	up{job="campuseats-api"} == 0 for 15 s	critical
HighServerErrorRate	more than 5 % of requests return 5xx for 1 min	warning
SlowApiResponses	p95 latency above 500 ms for 1 min	warning
KitchenBacklog	more than 10 orders waiting in PLACED for 1 min	warning

Table 5.6: Prometheus alert rules

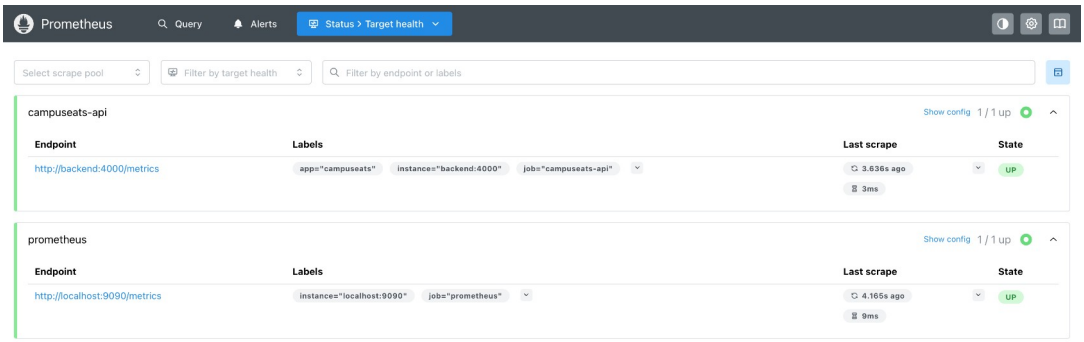


Figure 5.2: Prometheus scrape targets, both UP



Figure 5.3: Alert rules loaded and inactive under normal load

5.7.2 Grafana dashboard

Grafana is provisioned from files in the repository, so the datasource and the “CampusEats – Live Operations” dashboard exist as soon as the container starts, with no manual setup. The upper row answers the canteen’s questions—orders and revenue in the last hour, orders waiting for the kitchen, average preparation time—while the lower row answers the operator’s: whether the API is up, request rate, error rate, latency percentiles and resource use. Figures 5.4 and 5.5 were captured during the simulated lunch rush; the surge in orders per minute and the kitchen queue building up in the Preparing stage are both visible.

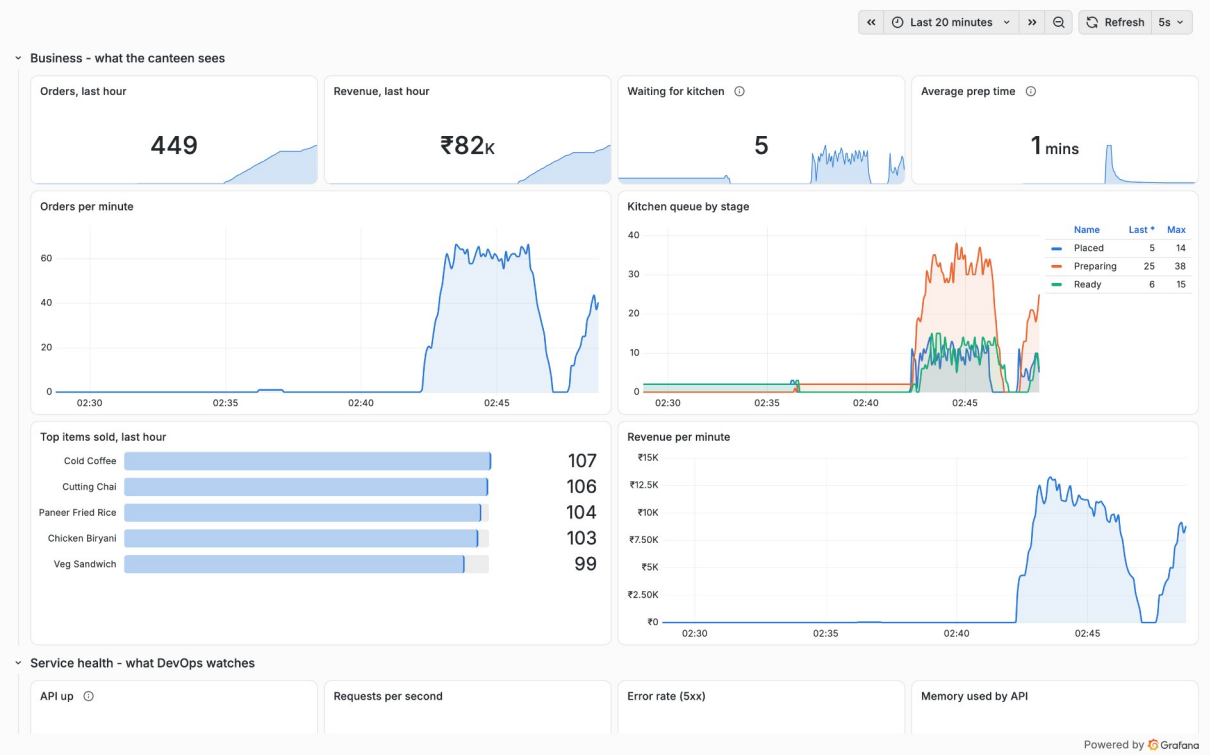


Figure 5.4: Grafana — business row during the lunch rush

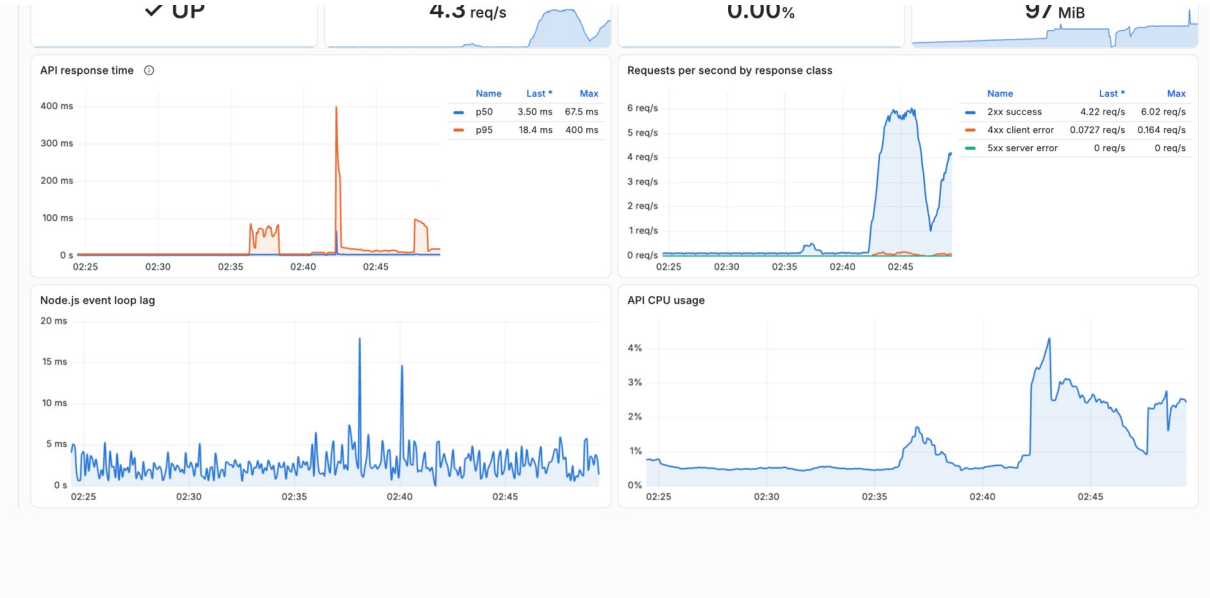


Figure 5.5: Grafana — service health row

5.7.3 Nagios

The Nagios container joins the Compose network, so it addresses each container by its service name. The object definitions in `monitoring/nagios/campuseats.cfg` describe four container hosts—with the backend declared as a child of MySQL and Redis, so that a database failure is reported as the root cause rather than as an API failure—and the Mac running the stack, which is queried through the NRPE agent installed in Experiment 7.

Host	Service	Check
campuseats-frontend	Web app HTTP	check_http port 80, /nginx-health must contain “ok”
campuseats-backend	API health	check_http port 4000, /api/health must contain “UP”
campuseats-backend	Menu endpoint	check_http port 4000, /api/menu must contain “price”
campuseats-mysql	MySQL port 3306	check_tcp
campuseats-redis	Redis port 6379	check_tcp
Mac_Host_001	PING, Jenkins port 8080	check_ping, check_tcp
Mac_Host_001	Root Partition, CPU Load, Total Processes, Current Users	check_nrpe → agent on port 5666

Table 5.7: Nagios hosts and services

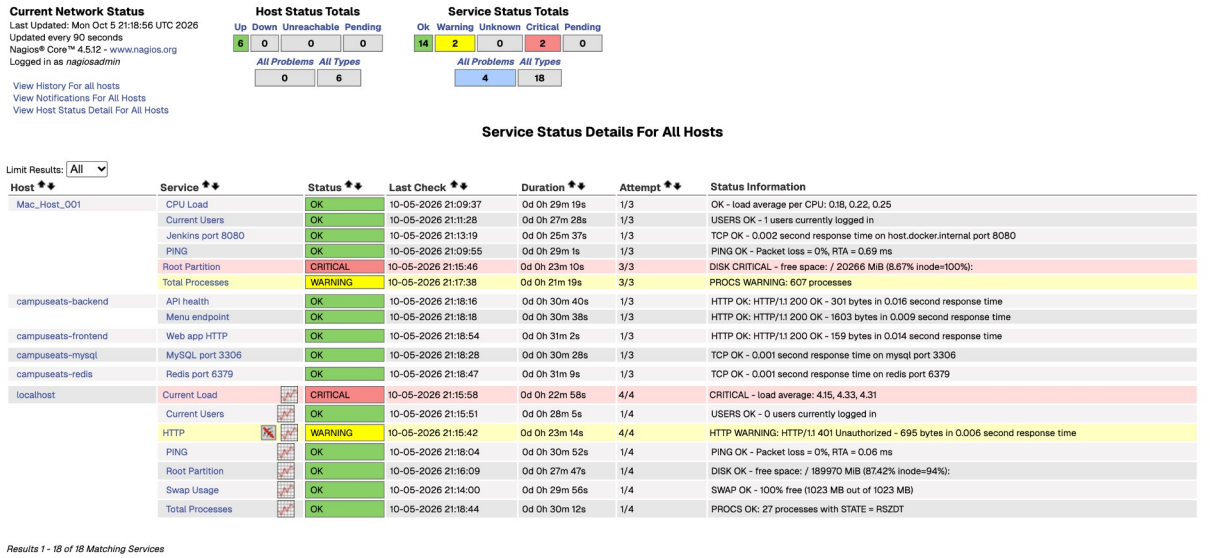


Figure 5.6: Nagios service status for all hosts

All application checks are OK in Figure 5.6. The non-OK states are genuine readings, not configuration faults. The Mac’s root partition is CRITICAL because the disk had less than ten per cent free space; this is a real problem that the check exposed during the project. Total Processes is in WARNING because the browser used for screenshots and the load simulator pushed the count above the threshold of 600. The localhost HTTP warning and load critical belong to the configuration bundled with the image, which monitors the container itself: its HTTP check receives 401 because the web interface is password-protected, exactly as observed in Experiment 7, and its load figure includes the overhead of running an amd64 image under emulation.

6. Testing

6.1 Automated Tests

API tests use Supertest to send real HTTP requests to an Express application built with the in-memory store, so each test exercises routing, authentication, validation and business rules together while still running in milliseconds. A fresh application and store are created for every test, which keeps tests independent. The frontend tests cover the cart arithmetic, formatting and axis-tick logic that the pages depend on.

Suite	Tests	What is verified
auth.test.js	8	Registration, duplicate email, input validation, wrong password, /me, forged tokens
menu.test.js	8	Cache HIT/MISS, admin CRUD with cache invalidation, price validation, role limits, sold-out hiding
orders.test.js	16	Server-side totals, token sequence, line merging, bad carts, sold-out, state machine, ownership, cancellation, history
stats.test.js	6	Today vs yesterday, cancelled excluded, IST day and hour bucketing, top items, admin-only access
ops.test.js	4	Health UP and DOWN (503), /metrics content, JSON 404
lib.test.js (frontend)	8	Cart add/remove/limit/immutability, rupee format, token format, time ago, axis ticks
Total	**50**	

Table 6.1: Automated test suites

Measure	Statements	Branches	Functions	Lines
API coverage (Jest)	92.92 %	82.53 %	90.47 %	95.77 %

Table 6.2: Code coverage of the API

6.2 Deployment Smoke Tests

Unit and integration tests prove the code; the smoke test proves the deployment. After every release on `main`, the pipeline confirms that the API reports the new version and that the frontend and API respond through Nginx. All nine releases passed this check.

6.3 Load Simulation

The script `scripts/lunch-rush.mjs` simulates a lunch break: a configurable number of students register and place random orders every 8 to 25 seconds, about one request in twelve being deliberately invalid, while a simulated kitchen advances each order through the states with realistic delays. A four-minute run with 18 students produced the results below, read from Prometheus and the simulator.

Measure	Value
Orders placed	245 in 4 minutes

Measure	Value
Invalid orders rejected (expected 4xx)	24
Orders taken through PREPARING → READY → COLLECTED	245 / 245
Peak request rate	about 6 requests per second
Median (p50) API latency	≈ 3.5 ms
95th percentile latency	≈ 18 ms steady; one 400 ms spike at the start of the run
Server errors (5xx)	0

Table 6.3: Lunch-rush simulation results

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Student ordering, token, tracking	Delivered (CE-1, CE-2, CE-3, CE-8, CE-9)
Kitchen queue and stock	Delivered (CE-4)
Admin menu and sales	Delivered (CE-10, CE-11)
Scrum planning with points and DoD	15 issues, 32 story points, 2 sprints in Jira project CE, Definition of Done linked to builds
Branch per issue, PR-only merges	15 pull requests; every commit carries its issue key
Containerised stack, one command	7 services; docker compose up -d starts everything
Automated CI on every branch	19 builds across 11 branches, all SUCCESS, typically 17–24 s
Automated deployment with rollback	9 releases (1.0.1–1.0.9) deployed and smoke-tested automatically
Monitoring over time and of state	Prometheus (4 alerts) + Grafana (18 panels) + Nagios (18 services, 6 hosts)

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The clearest demonstration of the practices working together came after release 1.0.8. `docker compose ps` reported the frontend container as unhealthy although the application served every request. Inspection showed that inside the Alpine image `localhost` resolves first to the IPv6 address `::1`, while Nginx listened only on IPv4, so the health check's `wget` was refused every time. The defect was raised as bug CE-15, fixed on branch `fix/CE-15-frontend-healthcheck` by pointing the check at `127.0.0.1`, built and tested by Jenkins, merged as pull request #15 and deployed automatically as release 1.0.9, after which the container reported healthy. From detection to a deployed fix took one short iteration of the same loop used for features.

7.3 Challenges and Resolutions

Challenge	Resolution
Feature-branch builds overwrote release image tags	Separate tag schemes for main (1.0.n) and branches (0.0.n-branch)
Jenkins on localhost cannot receive GitHub webhooks	Multibranch scan every minute; result posted back to the PR as a commit status
Jenkins runs as a launchd service with a minimal PATH	PATH set explicitly in the Jenkinsfile environment block
Private repository without stored Jenkins credentials	Git on the host authenticates through the GitHub CLI credential helper
Port 5000 reserved by macOS AirPlay; 3306 and 8080 already in use	API on 4000, MySQL on 3307, Nagios on 8090

Challenge	Resolution
Nagios image is amd64-only; checks slower under emulation	Platform pinned in Compose; HTTP thresholds widened to 2 s / 5 s
Nagios rejects “(” and “)” in service names	Service descriptions renamed; configuration validated with nagios -v before reload
Frontend health check failed on IPv6 localhost	Bug CE-15, fixed through the pipeline (Section 7.2)

Table 7.2: Challenges met and how they were resolved

7.4 Limitations

- All work was carried out from one machine and one GitHub account. GitHub does not allow an account to approve its own pull request, so formal review approvals were not recorded; the Jenkins status check served as the merge gate. With individual accounts, branch protection would require one approving review in addition to the green check.
- Jenkins, the application and the monitoring stack share one host, so a host failure would take down both the system and the means of observing it. A production setup would separate them.
- Prometheus alerts are evaluated and displayed but not routed to email or chat, as no Alertmanager is deployed.
- Live updates use polling every three seconds, which is simple and robust but less efficient than server-sent events or WebSockets at larger scale.

8. Conclusion and Future Scope

8.1 Conclusion

CampusEats meets its functional goal: a student can order from class, receive a token, follow the order through the kitchen and collect it when it is ready, while the kitchen works a live queue and the administrator manages the menu and studies demand. More importantly for this course, the project shows the seven lab experiments operating as one system rather than as separate exercises.

Planning the work in Jira produced issue keys that then appeared in every branch, commit, pull request and build, which made the path from a requirement to the code that satisfies it and the release that delivered it traceable in both directions. Docker and Docker Compose made the environment a versioned artifact, so the stack that Jenkins deployed was the stack that developers ran. The multibranch pipeline turned every push into evidence about the change, and the version-aware smoke test and rollback made deployment routine: nine releases went out without a manual step. Monitoring closed the loop, both by showing the system's behaviour under a simulated rush and by revealing a real defect that was then fixed through the same flow. The practical lesson repeated across the project is that each practice is most valuable for what it enables in the next one.

8.2 Future Scope

- Online payment through UPI with payment status driving the order state.
- Push or SMS notification when an order becomes READY, replacing polling on the tracking page.
- Preparation-time estimates per dish from the recorded `prep_seconds` history, shown to students before ordering.
- Multiple canteens and counters, with the queue partitioned by counter.
- Deployment to a Kubernetes cluster with rolling updates, and an Alertmanager routing Prometheus alerts to the team.
- Branch protection with required reviews and the Jenkins check once every member works from their own account.